

App Dev Project Report: AarohanSetu

1. Student Details

- **Name:** Shambhavi Shekhar
- **Roll Number:** 23f2000348
- **Email:** 23f2000348@ds.study.iitm.ac.in
- **About Me:** I am a diploma-level student in the IIT Madras BS Degree program. I have a deep curiosity for data-driven backend systems and web technologies. Beyond software development, I am highly interested in machine learning, particularly in exploring how predictive models and recommendation algorithms can be applied to user interaction logs to automate scheduling patterns and build smarter, more personalised user experiences.

2. Project Details

- **Project Title:** AarohanSetu (Placement Management Application V2)
- **Problem Statement:** To design and build a multi-role, responsive web application for managing university placement operations. The application coordinates:
 1. **Administrators:** To manage user accounts, approve corporate partners, monitor placement stats, and blacklist or deactivate profiles.
 2. **Recruiting Companies:** To update recruiter profiles, create placement drives, manage rosters, schedule interviews, and select/reject candidates.
 3. **Students (Users):** To browse and filter available drives based on academic eligibility, apply for jobs, upload resumes, and view past placement history. The app must run with high responsiveness under cache invalidation and handle background reporting/notification tasks.
- **Approach:** The system is built as a decoupled Single Page Application (SPA):
 - **Backend:** Flask REST API providing session-based authentication, automated eligibility verification, database transactional integrity, and modular schema representations.
 - **Frontend:** Vue 3 serving as a dynamic, responsive client interface using pure Bootstrap 5 layouts, custom glassmorphism components, and Chart.js integration.
 - **Asynchronous Tasks & Optimization:** Redis server serving as both a key-value caching layer (with invalidation on modifications) and a broker for Celery worker processes handling daily reminders, monthly reports, and history exports.

3. AI/LLM Declaration

I used code-assistance tools to scaffold the frontend Vue.js Single File Components, write custom CSS visual styles, and ensure grid responsiveness under Bootstrap 5.

- **Extent of usage:** The extent of AI/LLM usage is around 30%, limited to frontend styling, responsive UI card design suggestions, and initial Vue form structures.
- **Manual implementation:** All backend Flask API handlers, database model schema definitions, and test suites were debugged, and integrated manually.

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend web framework serving RESTful API routes.
VueJS (Core + Router)	Reactive frontend Single Page Application (SPA) structure and routing.
SQLite + SQLAlchemy	Relational database engine and Object Relational Mapper (ORM) for data persistence.
Bootstrap 5	Responsive layout grid, buttons, and navigation bar styles.
Redis	Key-value caching storage (admin dashboard stats) and broker for backend Celery workers.
Celery	Asynchronous task queue handling daily reminders, monthly reports, and CSV exports.

5. Database Schema / ER Diagram

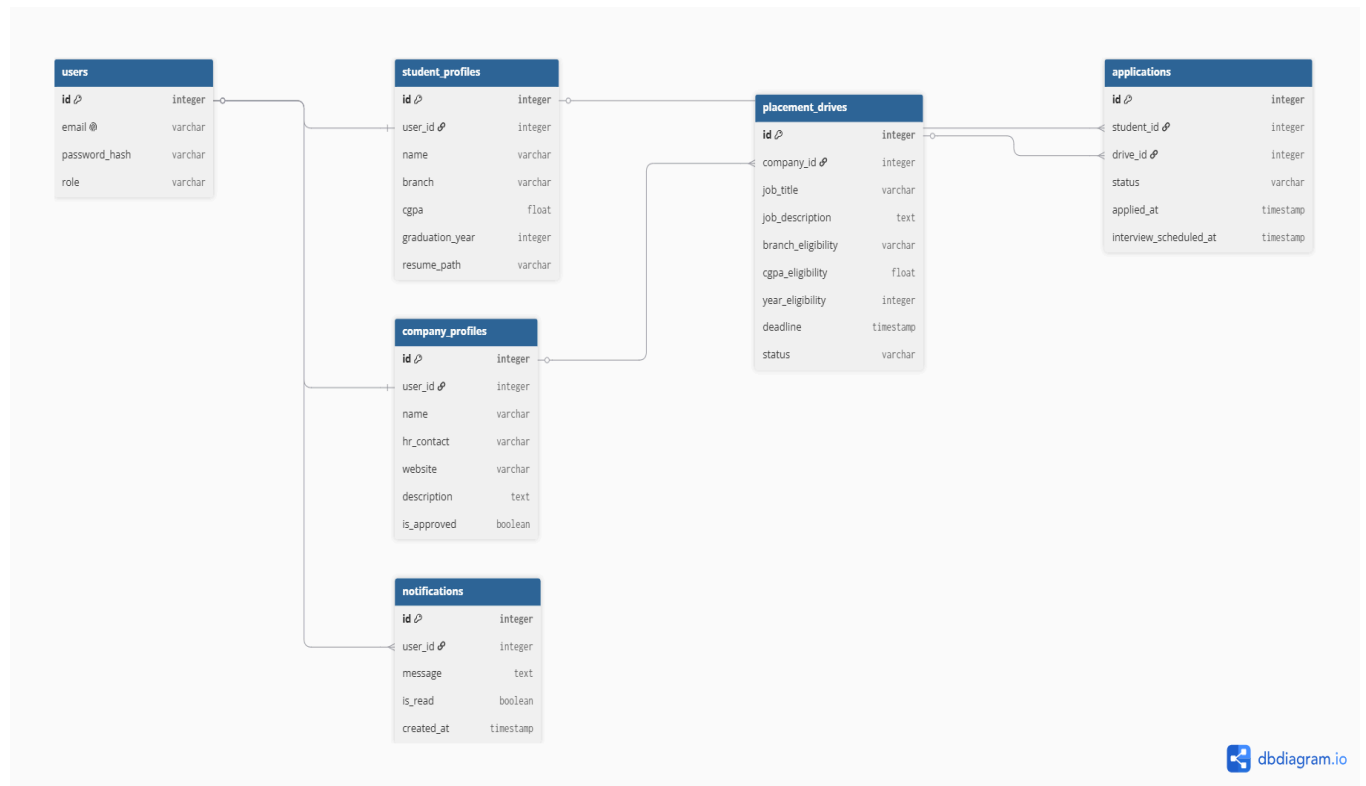
Tables:

1. **User (users):** Stores core credential data (id, email, password_hash, role).

2. **StudentProfile (student_profiles):** Stores student academic profiles (id, user_id, name, branch, cgpa, graduation_year, resume_path).
3. **CompanyProfile (company_profiles):** Stores recruiter profile records (id, user_id, name, hr_contact, website, description, is_approved).
4. **PlacementDrive (placement_drives):** Stores job drive details (id, company_id, job_title, job_description, branch_eligibility, cgpa_eligibility, year_eligibility, deadline, status).
5. **Application (applications):** Tracks student job applications (id, student_id, drive_id, status, applied_at, interview_scheduled_at).
6. **Notification (notifications):** Stores system notification logs (id, user_id, message, is_read, created_at).

Relationships:

- **One-to-One:** User ↔ StudentProfile (linked via user_id foreign key).
- **One-to-One:** User ↔ CompanyProfile (linked via user_id foreign key).
- **One-to-Many:** CompanyProfile ↔ PlacementDrive (linked via company_id foreign key).
- **One-to-Many:** StudentProfile ↔ Application (linked via student_id foreign key).
- **One-to-Many:** PlacementDrive ↔ Application (linked via drive_id foreign key).
- **One-to-Many:** User ↔ Notification (linked via user_id foreign key).



6. API Resource Endpoints

Endpoint	Method	Description
/api/login	POST	Authenticates users (unified) and generates a session.
/api/register/student	POST	Self-registers a new student profile account.
/api/register/company	POST	Self-registers a new company profile account.
/api/logout	POST	Ends the user session.
/api/admin/stats	GET	Returns cache-optimized global placement stats.
/api/admin/companies	GET / POST	Admin controls to view and approve/reject company registrations.
/api/admin/drives	GET / POST	Admin controls to view and approve/reject placement drives.
/api/student/drives	GET	Returns eligible job drives list based on student parameters.
/api/student/apply	POST	Submits job application (protected against duplicates/eligibility).
/api/student/resume	POST	Uploads student resume.
/api/student/export-csv	POST	Triggers async Celery job to export application history to CSV.
/api/company/drives	POST	Recruiters control to propose new placement drives.

Endpoint	Method	Description
/api/company/applications/<id>/status	POST	Recruiters control to shortlist, select, or reject applications.
/api/company/applications/<id>/schedule	POST	Recruiters control schedule interview timings.

7. Architecture and Features

Architecture Overview:

- **run.py** – Main entry point booting the Flask application and dynamic Redis checks.
- **backend/**
 - **app.py** – Core initialization container (Blueprints, database, cache registration).
 - **config.py** – Environment definitions, Redis configuration, and folder setup.
 - **models/db_models.py** – Declarative SQLAlchemy models defining tables and relations.
 - **routes/** – Modular Flask controllers (auth.py, admin.py, company.py, student.py).
 - **tasks.py** – Background Celery tasks logic (reminders, HTML reports, CSV exports).
- **frontend/** – Vue 3 client Single Page Application.

Implemented Core Features:

1. **Unified Authentication & Role Gates:** All accounts are verified on a single User table using session management.
2. **Strict Eligibility Checks:** Student academic properties (branch, CGPA, graduation year) are validated programmatically before job application submission.
3. **Active Cache Expiry:** Admin dashboard stats are cached in Redis and dynamically invalidated on database updates (such as student or company approvals).

Additional Features:

1. **Daily deadline reminder emails:** Scheduled background jobs notifying students of upcoming application deadlines.
2. **User-triggered asynchronous CSV exports:** Offloaded Celery task that saves applicant histories and notifies the student with a download link.

3. **Interactive Stats Charts:** Embedded Chart.js rendering stats and counts on the Admin dashboard.

8. Video Presentation

- **Drive Link:**

 MAD 2.mp4